

Zhuocheng Qiao

AI Agent / Backend Software Engineer

Shanghai | M.S. in Signal and Information Processing, East China Normal University (2022)

18017822420 | 18017822420@163.com | github.com/tomqiaozc



SUMMARY

Software Engineer with 4+ years at Microsoft, formerly Software Engineer II, focused on production multi-agent systems, Agent Harnesses, and enterprise backend platforms. Experienced with Python, .NET, and Azure across Agent Runtime, Context/Memory, tool and sandbox execution, reliable messaging, service integration, security, and commercial systems. Particularly focused on trustworthy execution for probabilistic Agents—using deterministic control, independent evaluation, evidence gates, and actual test execution to close the gap between claimed and verifiable completion.

TECHNICAL SKILLS

Agent & AI Systems: Multi-Agent / Sub-Agent, Native Function Calling, Context/Memory, Agentic RAG, Agent Harness, DAG workflows, independent evaluation and evidence gates, browser and code execution

Backend & Distributed Systems: Python, FastAPI, C#/.NET, ABP Framework, REST/SSE, Service Bus, Redis, asynchronous processing, idempotency, state machines, external-system integration

Cloud & Engineering: Azure, Azure AI Search, Blob Storage, Key Vault, Docker, CI/CD, observability, automated testing, reliability and security engineering

PROFESSIONAL EXPERIENCE

Microsoft | Software Engineer II

Jun 2022–Aug 2026

Microsoft Office Agent | Production Multi-Agent System

Jul 2025–Feb 2026

Production general-purpose Agent for M365 users, supporting complex planning, concurrent Sub-Agents, tool and code execution, local-browser operations, long-term memory, and Office deliverables. Built on Azure across Agent Runtime, sandbox, data, and messaging infrastructure; reached **27K WAU** about two months after global launch. As a core contributor, focused on tool protocols, long-running task reliability, and Agent service integration, while contributing to Context/Memory, sandbox execution, and security.

- **Multi-Agent Runtime & Tool Calling:** Led the migration of main-Agent and Sub-Agent tool protocols from text XML to Native Function Calling. Adapted OpenAI/Anthropic streaming protocols and unified tool registration, argument aggregation and validation, backend execution, and Tool Result feedback. Designed Native/XML dual-mode rollout and fallback, and fixed incomplete streaming arguments, corrupted message sequences, and Tool Call/Result mismatches caused by process interruption.
- **Context, Memory & Agentic RAG:** Core contributor to Agent context and long-term memory. Added pre-call token accounting, hierarchical compaction, and dynamic Skill loading/unloading while deterministically preserving plans and tool state. Contributed to the architecture and core development of knowledge-base Agentic RAG, combining keyword, vector, and semantic ranking into a loop of candidate retrieval, evidence reading, continued search, and source citation.
- **Sandbox & Browser Execution:** Contributed to a Cloud Sandbox + Local Browser execution model. Coordinated multi-instance resources through warm pools, lifecycle management, caching, and distributed locks. Built DOM-first browser operation that reused local authenticated sessions and re-observed pages after actions to verify outcomes in real enterprise systems.
- **Long-Running Task Reliability:** Independently diagnosed that the original real-time channel lacked offline retention, acknowledgement, and redelivery. Defined retention, Ack, redelivery, and session-ordering semantics; abstracted a rollout-safe, reversible MessageTransport, introduced a reliable ordered channel, and retained replayable process state for backend disruption and browser reconnection recovery.
- **Service Integration & Security:** Independently delivered Agent Proxy from zero to production, covering delegated downstream authorization, asynchronous task persistence and restart recovery, containerization, and CI/CD. Packaged Agent Runtime as a production service for external integration; designed user-scoped encryption and progressive rotation, certificate injection, inter-service HTTPS, and downstream certificate validation.

Microsoft Copilot | Production Agent Harness & Loop Engineering

Mar 2026–Aug 2026

Contributed to building and scaling a multi-Agent Harness for real software delivery; it became the primary execution path for project requirements. External Issues became persistent Tasks, while a state graph drove Developer, Reviewer, and Judge through implementation, review, and acceptance. Task claiming and timeout takeover, retries, and failure recovery kept long-running work reliable; code, tests, PRs, CI, reviews, and screenshots governed state transitions and merge decisions.

- **Graph-Based Harness:** Modeled implementation, independent review, evidence adjudication, repair iteration, and merge as a reversible DAG centered on Task State. Each Agent handled only its current-state responsibility, with persistent state and artifacts carrying work across runs rather than relying on one model context for the full workflow.
- **Loop Engineering Flywheel:** Aggregated traces, failure types, duration, retries, architecture checks, and delivery evidence across tasks to identify recurring systemic issues. Fed conclusions back into task templates, Skills, rules, tests, or the Harness, then used later tasks to verify whether each improvement worked—forming an execute, aggregate, write-back, and re-validate loop.

PROFESSIONAL EXPERIENCE (CONTINUED)

Office Plus | Commercial Platform & Identity Infrastructure

Jul 2024–Jun 2025

Multi-service commercial platform built with .NET/ABP and Azure, covering orders, payments, membership and paper entitlements, growth campaigns, and identity. Supported **3.4M new paid subscriptions and US\$6.9M in FY25 revenue**. As a core backend contributor, led third-party capability productization, recurring subscriptions, partner fulfillment, and identity governance.

- **Third-Party Productization & Asynchronous Delivery:** Led end-to-end commercialization of a third-party plagiarism-checking service, expanding an API into a complete flow for upload, payment, word-count entitlements, asynchronous detection, result delivery, and refund on failure. Designed PaperRecord, PaperRight, and consumption mappings; used direct Blob upload, Service Bus workers, conditional database updates, and result MD5 validation to handle large files, entitlement oversell, rate limits, and compensation. Processed **6,320 orders in the first 10 days**.
- **Subscriptions, Payments & Partner Fulfillment:** Led WeChat recurring billing and Partner VIP direct fulfillment while contributing to the broader order and payment platform. Layered Contract, Order, Payment, and entitlement models; used state validation, unique business keys, and retryable compensation against duplicate charges, duplicate fulfillment, and partial cross-service failures. Combined App Roles, partner signatures, and external-order idempotency, while separating fulfillment and notification states. Auto-renewal SKUs reached **86%** of orders and ARPPU increased **58% MoM**.
- **Identity Sessions & Account Lifecycle:** Led redesigns of Refresh Token, device sessions, and soft deletion. RSA JWT and UserDeviceRecord enabled device-level revocation, quotas, and gradual legacy-token migration, changing token-record growth from daily accumulation to convergence by active device. Designed a recoverable deletion state machine, durable events, and periodic reconciliation; under high-frequency refresh baselines and million-message load tests, used synchronous keep-alive with asynchronous batch cleanup to reduce online database load.

Earlier projects: XPay (Jun–Oct 2022) | PC Manager (Nov 2022–Jun 2024)

PERSONAL ENGINEERING & OPEN SOURCE

Graph Rail | Graph-Based Agent Harness

Built an Agent engineering Harness for complex software tasks, organizing Build, Review, Test, and Gate nodes as a directed graph. Added deterministic process control, independent evaluation, and evidence verification around probabilistic Agents to close the trust gap between an Agent claiming completion and a task being verifiably complete.

- **Independent Evaluation:** Applied the principle that the implementer does not evaluate itself. Implementer, Reviewer, and Test Designer are isolated-context Agents; Test Executor is not delegated to an Agent—the Harness process runs test commands and binds the evidence, preventing “the Agent says it ran the tests.”
- **Test Authenticity:** The Harness does not treat an Agent-generated report as proof of execution. It runs the commands itself and binds test plan, command, and result hashes with an HMAC chain; it rejects evidence without provenance, self-authored evidence, post-execution edits, and invalid “0 tests passed” results.
- **Deterministic Control Plane:** Modeled the workflow as a directed graph with PASS / ITERATE / FAIL edges. Handshakes, legal state transitions, mechanical gates, fail-closed validation, and loop limits constrain the Agents so progression, rework, and termination do not depend on model self-reporting.

Open Source | Hermes Agent / Multica

Contributed to Agent Runtime security, tool-call reliability, and multi-tenant data boundaries. Multiple Hermes Agent implementations entered mainline through maintainer cherry-pick/salvage with authorship retained; five Multica PRs were merged directly.

- **Hermes Agent | Credential Lifecycle Security:** Found that the Gateway persisted short-lived pairing codes in plaintext, allowing local file readers to bypass the interactive flow. Designed random-salt SHA-256 storage, constant-time comparison, legacy-format migration, and invariant tests for irrecoverable plaintext, salt uniqueness, and old/new interoperability; the implementation entered mainline after maintainer integration ([#8056](#)).
- **Hermes Agent | Streaming Tool-Call Integrity:** Found that some API routers rewrote `finish_reason`, allowing truncated JSON arguments to bypass length checks and reach execution. Moved validation from unreliable completion metadata to Tool Call structure, independently checking closure and parseability before execution. The submission was cherry-picked into mainline ([#7693](#)).
- **Multica | Multi-Tenant & Content-Security Boundaries:** Drove five merged PRs, moving Workspace validation from handlers into SQL DELETE / UPDATE predicates and locking the invariant with cross-tenant negative tests. Also fixed Stored XSS from inline SVG rendering by forcing untrusted content to download at the shared-storage boundary, providing defense in depth when callers omit validation ([#3027](#), [#3023](#)).